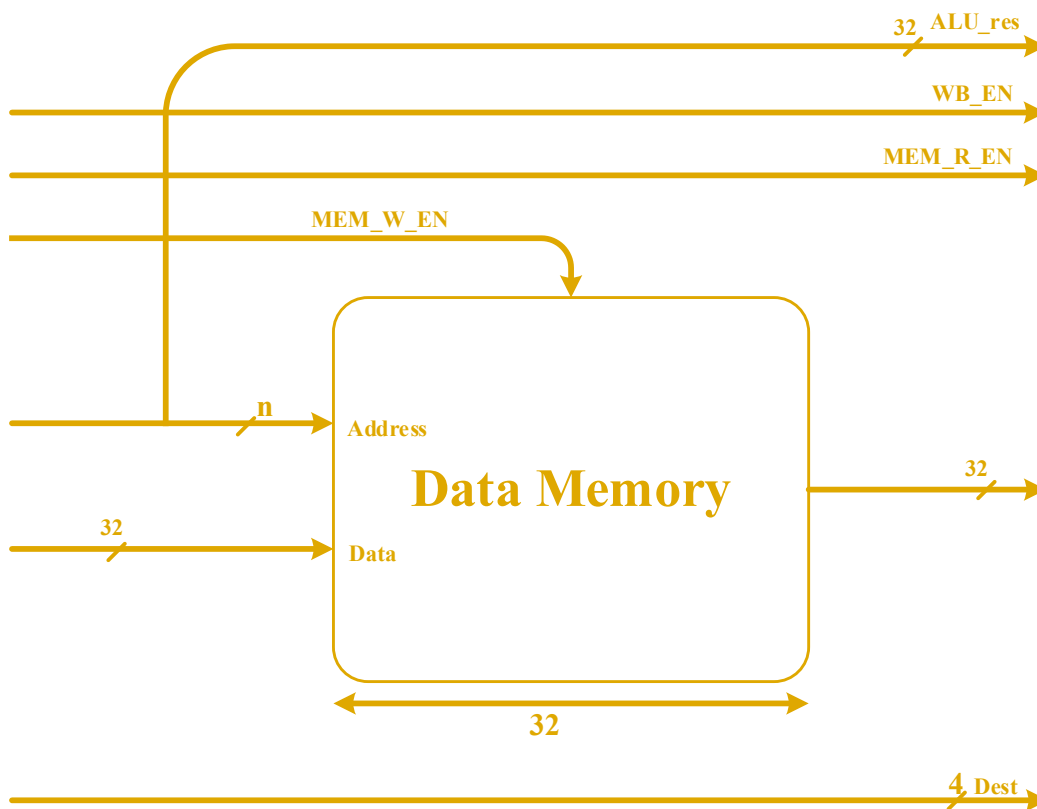




قسمت سوم آزمایش پیاده سازی ARM (جلسه چهارم):

در جلسات گذشته، مراحل واکشی، کدگشایی و اجرا پردازنده پیاده سازی گردید. در این جلسه، ابتدا مرحله‌ی حافظه و مازول تشخیصی مخاطره را پیاده سازی کنید، سپس کل پردازنده را تست کنید.

پیاده‌سازی مرحله حافظه (Memory Stage)



شکل ۱



آزمایش دوم: پیاده سازی پردازنده ARM

گرد آورنده: امید قلی زاده، امیررضا غلامی

برای پیاده سازی حافظه داده (Data Memory)، همانند حافظه دستور العمل (Instruction Memory)، از هسته ی آماده ی Vivado استفاده کنید. تنظیمات این حافظه را به صورت زیر تعیین کنید:

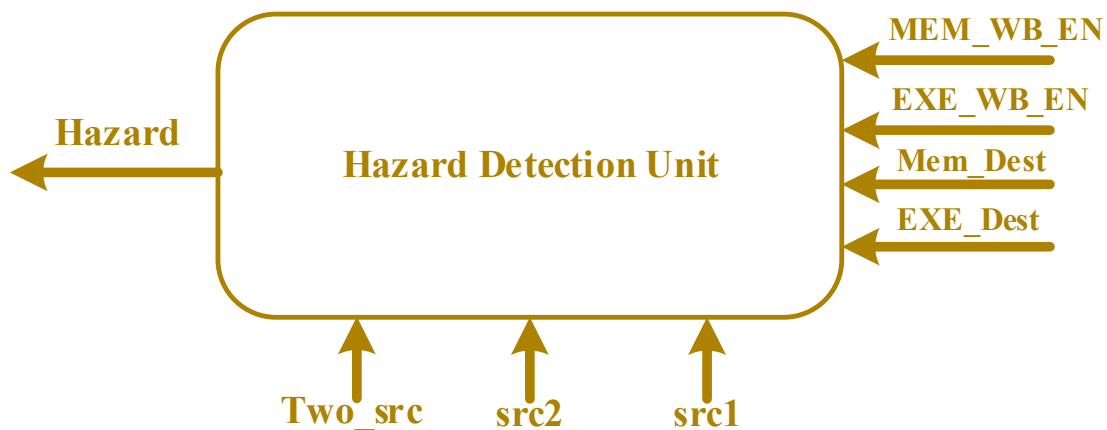
- Data Width (عرض داده): ۳۲ بیت.
- Depth (عمق): این متغیر، عرض بیت آدرس مموری را نیز تعیین می کند. یک m مناسب انتخاب کنید و یک slice به اندازه $n = \log_2(m)$ از خروجی ALU رو جدا کرده و به عنوان آدرس به ALU دهید.
- Memory Type: گزینه ی Single Port RAM را انتخاب کنید.
- مقداردهی اولیه: در تب RST & Initialization، در تکست بار Coefficients file، عبارت no_coe_file_loaded را قرار دهید تا مقداردهی اولیه صورت نگیرد.
- در تب Port Config، در بخش های Input Option و Output Option، اطمینان حاصل کنید که گزینه Non Registered فعال باشد.

پیاده سازی مرحله بازنشانی (Write Back Stage)

این مرحله بسیار ساده است. یک مالتی پلکسر (MUX) ایجاد کنید که بر اساس سیگنال MEM_R_EN تصمیم بگیرد که چه داده ای باید به Register File برود:

- اگر $MEM_R_EN = 1$: داده ی خوانده شده از حافظه.
- اگر $MEM_R_EN = 0$: داده ی محاسبه شده توسط ALU.

واحد تشخیص مخاطره (Hazard Detection Unit)



شکل ۲



آزمایش دوم: پیاده سازی پردازنده ARM

گرد آورنده: امید قلی زاده، امیررضا غلامی

حال باید ماژول تشخیص مخاطره افزوده شود. اما قبل از پیاده سازی، به طور مختصر انواع مخاطره ها را بررسی می کنیم:

به طور کلی در پردازنده ها سه نوع مخاطره وجود دارد:

۱. مخاطره ساختاری (Structural Hazard)

- این مخاطره زمانی رخ می دهد که سخت افزار کافی برای اجرای همزمان تمام ترکیب های دستورات وجود نداشته باشد.
- در پردازنده ما، این مخاطره بین مرحله WB و ID به دلیل تلاش همزمان برای خواندن و نوشتن در Register File رخ می دهد.
- راه حل: برای رفع این مخاطره، نوشتن در ثبات های عمومی را به لبه پایین رونده کلاک و خواندن را به صورت آسنکرون (ترکیبی) تغییر دادیم. بنابراین این مخاطره در پردازنده ما رفع شده فرض می شود.

۲. مخاطره کنترلی (Control Hazard)

- این مخاطره ناشی از دستورات پرش (Branch) است.
- وقتی یک دستور پرش وارد خط لوله می شود، تا زمانی که آدرس مقصد پرش محاسبه نشود، پردازنده نمی داند دستور بعدی چیست. در این مدت، ممکن است دستورات اشتباهی وارد خط لوله شوند.
- راه حل: برای رفع این مشکل، مکانیزم Flush (تخلیه خط لوله) استفاده می شود که پس از قطعی شدن پرش، دستورات غلط وارد شده را حذف می کند.

۳. مخاطره داده ای (Data Hazard)

این مخاطره زمانی رخ می دهد که اجرای یک دستور به نتیجه دستور قبلی که هنوز کامل نشده، وابسته باشد. این مخاطره انواع مختلفی دارد:

a. خواندن پس از نوشتن (RAW - Read After Write):

رایج ترین نوع مخاطره است. زمانی رخ می دهد که یک دستور می خواهد داده ای را از رجیستری بخواند که دستور قبلی هنوز مقدار جدید آن را ننوشته است. مثال:

`SUB R2, R0, R1` ; R2 محاسبه می شود اما هنوز در WB نوشته نشده

`AND R3, R2, R1` ; R3 به مقدار جدید R2 نیاز دارد (RAW Hazard)

راه حل در این آزمایش: ما در این آزمایش با استفاده از واحد تشخیص مخاطره و ایجاد توقف (Stall) این نوع هازارد را برطرف می کنیم.



آزمایش دوم: پیاده سازی پردازنده ARM

گرد آورنده: امید قلی زاده، امیررضا غلامی

b. نوشتن پس از خواندن (WAR - Write After Read):

- زمانی رخ می دهد که یک دستور بخواند در رجیستری بنویسد که دستور قبلی هنوز خواندن از آن را تمام نکرده است.
- این مخاطره در پردازنده های In-Order (مانند پردازنده ما) رخ نمی دهد.

c. نوشتن پس از نوشتن (WAW - Write After Write):

- زمانی رخ می دهد که دو دستور بخواهند در یک رجیستر یکسان بنویسند و ترتیب نوشتن آن ها به هم بریزد.
- این مخاطره نیز در پردازنده های In-Order ما رخ نمی دهد.

برای رفع هازارد RAW، ماژول Hazard_Detection_Unit را همانند شکل ۲ به پردازنده اضافه نمایید.

در این واحد، منابع دستور فعلی در مرحله ID ($src1, src2$) با مقصدهای دستورات قبلی که در مراحل EXE و MEM هستند، مقایسه می شود. اگر برابری وجود داشته باشد، یعنی دستور فعلی به داده ای نیاز دارد که هنوز آماده نیست.

شرایط تشخیص هازارد (تولید سیگنال $Hazard_Detected = 1$):

شما باید تمامی حالاتی که هازارد RAW رخ می دهد را در نظر بگیرید:

۱. هازارد با دستور مرحله EXE:

- اگر $(src1 == EXE_Dest)$ و $(EXE_WB_EN == 1)$
- اگر $(src2 == EXE_Dest)$ و $(EXE_WB_EN == 1)$ و $(Two_src == 1)$

۲. هازارد با دستور مرحله MEM:

- اگر $(src1 == MEM_Dest)$ و $(MEM_WB_EN == 1)$
- اگر $(src2 == MEM_Dest)$ و $(MEM_WB_EN == 1)$ و $(Two_src == 1)$



اعمال توقف (Stall) به خط لوله:

وقتی Hazard_Detected یک شد، باید سه کار انجام دهید:

۱. توقف PC: رجیستر PC نباید آپدیت شود (سیگنال freeze آن فعال شود).
۲. توقف IF/ID: رجیستر میانی IF/ID نباید آپدیت شود (سیگنال freeze آن فعال شود) تا دستور فعلی دوباره دیکد شود.
۳. تزریق حباب (Bubble): باید از ورود دستور فعلی به مرحله بعدی (EXE) جلوگیری کنید. برای این کار، تمامی سیگنال های کنترلی که قرار است وارد رجیستر ID/EXE شوند را صفر کنید (معادل یک دستور NOP). این کار با استفاده از MUX روبروی Control Unit انجام می شود.

تست نهایی و دیباگ با Vivado ILA

- دستورات برنامه محک (Benchmark) را که شامل وابستگی های داده ای هستند، در حافظه دستورات قرار دهید.
- هسته ILA را آپدیت کنید و مقادیر رجیسترهای ۰ تا ۶ واقع در رجیستر فایل را نمایش دهید. مقادیر ذخیره شده در این رجیسترها در صورت مرتب بودن نشان می دهد که دستورات به درستی انجام شده اند یا خیر.
- در صورت درستی نتایج، آن ها را به دستیار آموزشی نشان دهید. لازم به ذکر است که در شبیه سازی، نحوه عملکرد سیگنال Hazard_Detected و ایجاد حباب در خط لوله نیز باید نمایش داده شود.